



Lists

Dr. Obed Noé Sámano Abonce

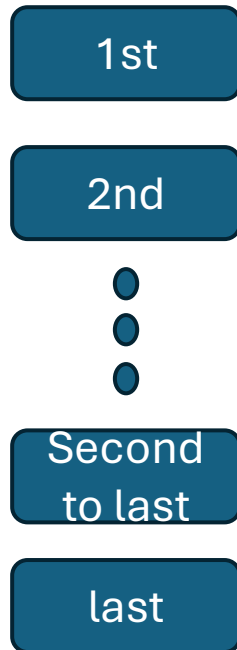


The List ADT

Definition

A **list** is a sequence of elements.

Each element has a position: there is a first, there is a last, and every element knows which one comes after it.



Operations

insert	Put an element in
remove	Take an element out
size	How many are there?
traverse	Visit every element once.
find	In what position is an element?

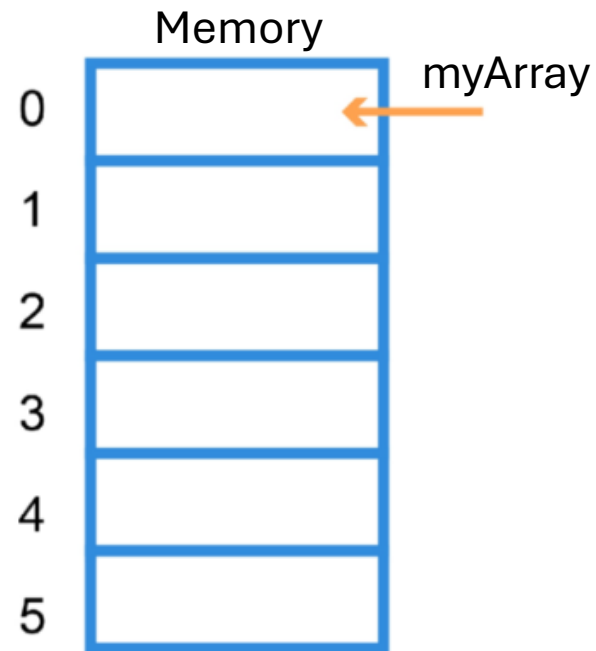
Other operations may be: printList, makeEmpty, next, previous, findKth

"There is no rule telling us which operations must be supported for each ADT; this is a design decision." — Weiss

The List ADT implemented as a vector

The list as a vector

Raw array vs vectors



	<code>int data[200];</code>	<code>vector<int> items;</code>
Size	fixed when you compile	grows while the program runs
Growing past capacity	you allocate a new one and copy by hand	it doubles its capacity for you
Knows its own size	no — you carry a separate count	yes — <code>items.size()</code>
Releasing the memory	<code>delete[]</code> , if you used <code>new</code>	automatic, in its destructor
Elements contiguous	yes	yes
findKth	$O(1)$	$O(1)$

The list as a vector

Example Interface

```
class ArrayList {  
    vector<int> items;    // the elements, in order  
public:  
    void insert(int pos, int x);  
    void remove(int pos);  
    int  find(int x);  
    int  findKth(int k);  
    void printList();  
    void makeEmpty();  
};
```

Same six operations. **The only thing decided here is that the elements sit next to each other.**

an array is not the list — it is one way to hold one

The list as a vector

Cost of the findKth & printList operations

Vectors shine in finding the kth element:



`findKth(5)` is at $1000 + 5 \times 4 = 1020$

`findKth`

one calculation, any size

$O(1)$

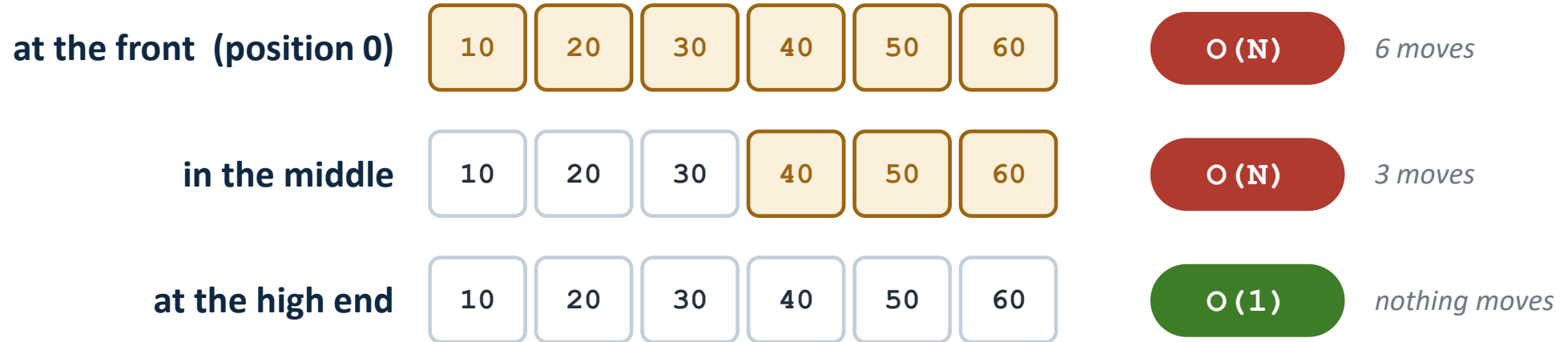
`printList`

touch every element once

$O(N)$

The list as a vector

Cost of the insert operation



on average, half the list moves — still linear

“If all the operations occur at the high end of the list... adding and deleting take $O(1)$ time... In such a case, **the array is a suitable implementation.**”

Weiss, §3.2.1

But if insertions and deletions happen **throughout the list, and especially at the front** — the array is not a good option.

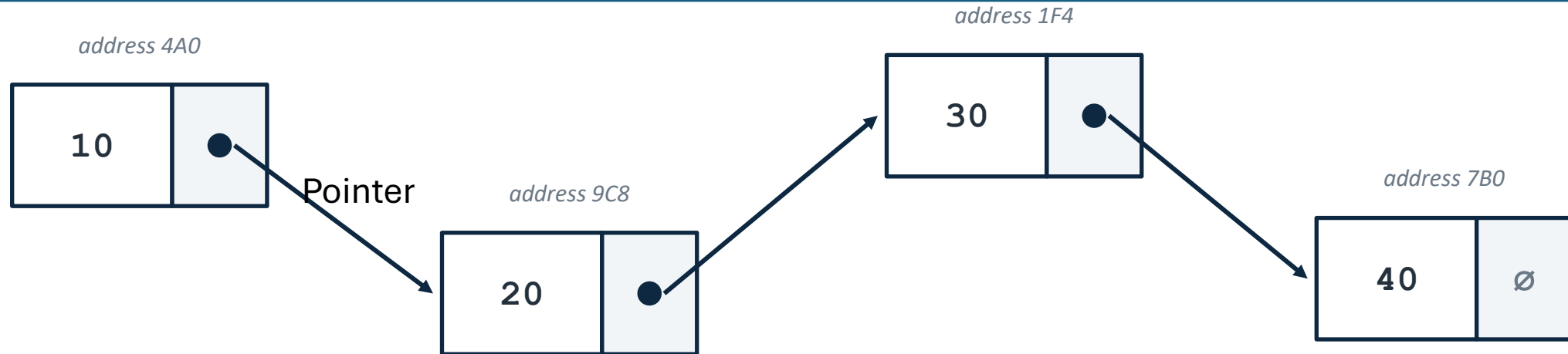
The List ADT implemented as a simple linked list

Simple Linked List

Intuition

*“In order to avoid the linear cost of insertion and deletion, we need to ensure that the list is **not stored contiguously**, since otherwise entire parts of the list will need to be moved.”*

Weiss, §3.2.2

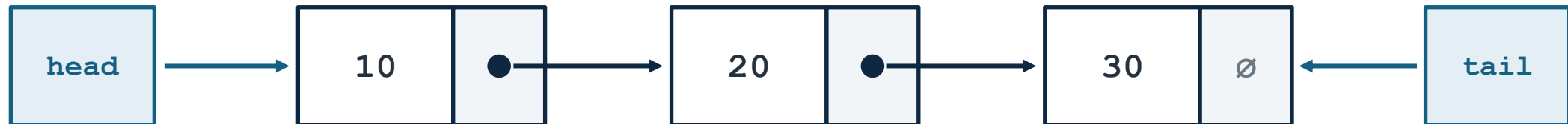


They are scattered. So each one has to record **where the next one is** — and that extra field is why we stop calling it an element and call it a **node**.

Simple Linked List

The Node

```
struct Node {  
    int    data;        // the element  
    Node *next;        // the link to the next node  
};
```

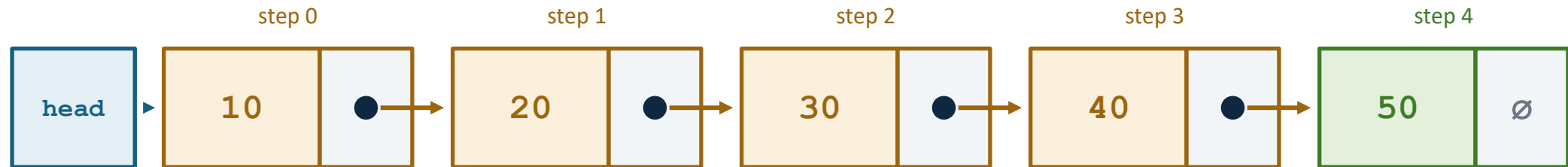


*“Thus, a typical linked list keeps **links to both ends of the list.**”* Weiss, §3.2.2

The last node’s next link points to **nullptr** — that is how the list says “this is the end”.

Simple Linked List

Cost of the findKth and printList operations



findKth(4) — four links followed before you arrive

`findKth(i)`

walk i links — the array did this in one step

$O(i)$

`printList / find`

same as the array, but a larger constant

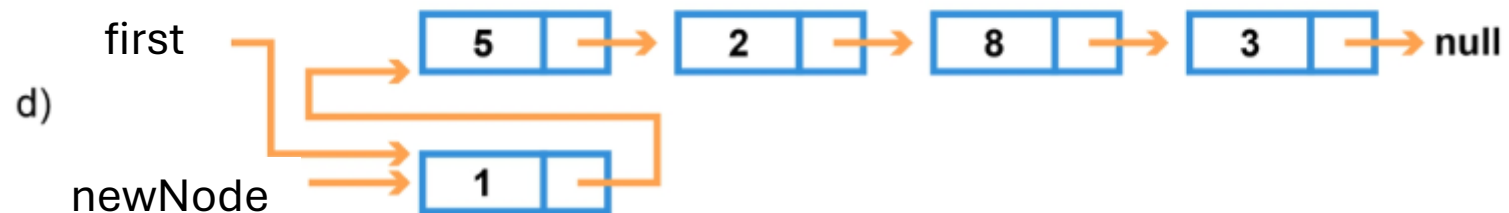
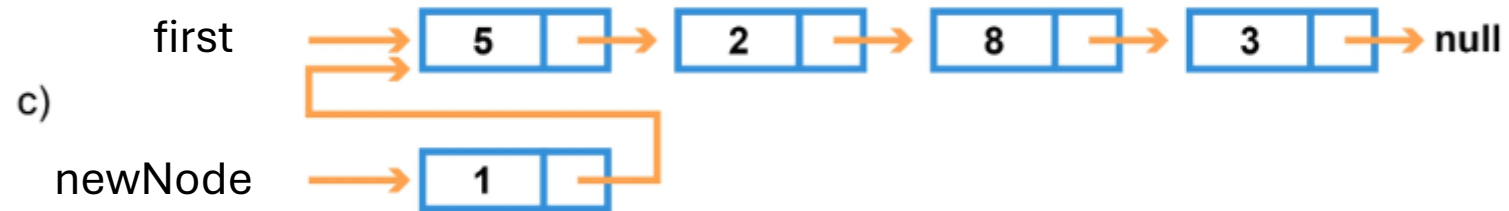
$O(N)$

Weiss is precise here: `findKth(i)` is $O(i)$, not $O(N)$ — and calls in sorted order (2, 3, 4, 6) can share **one scan**.

The superpower is gone. There is no arithmetic that finds a scattered node.

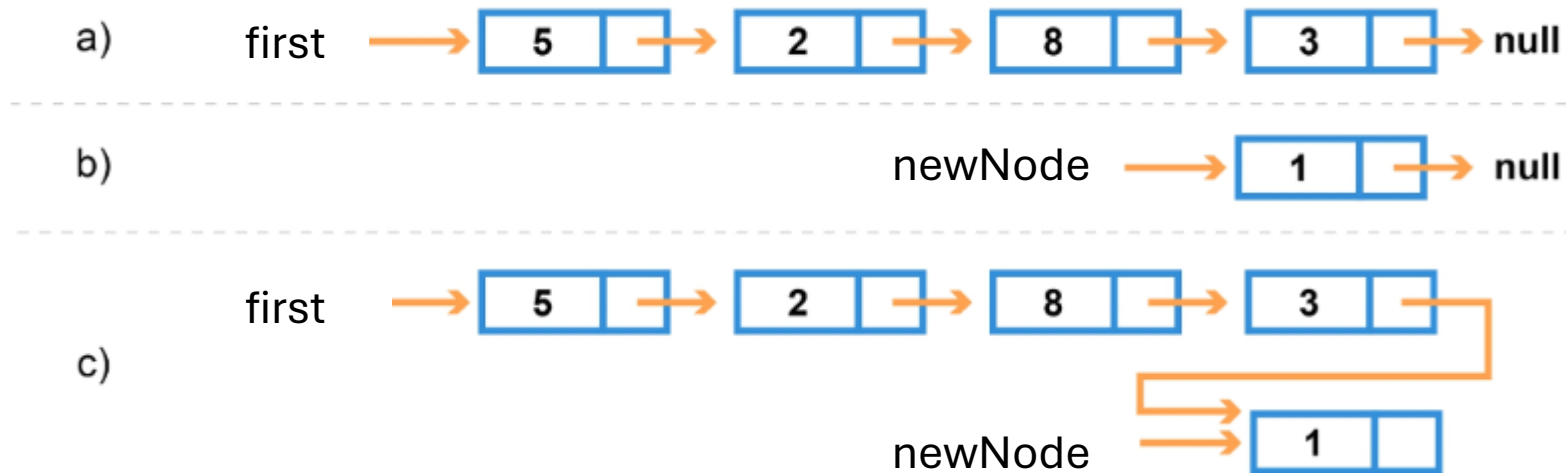
Simple Linked List

Add an element at the front



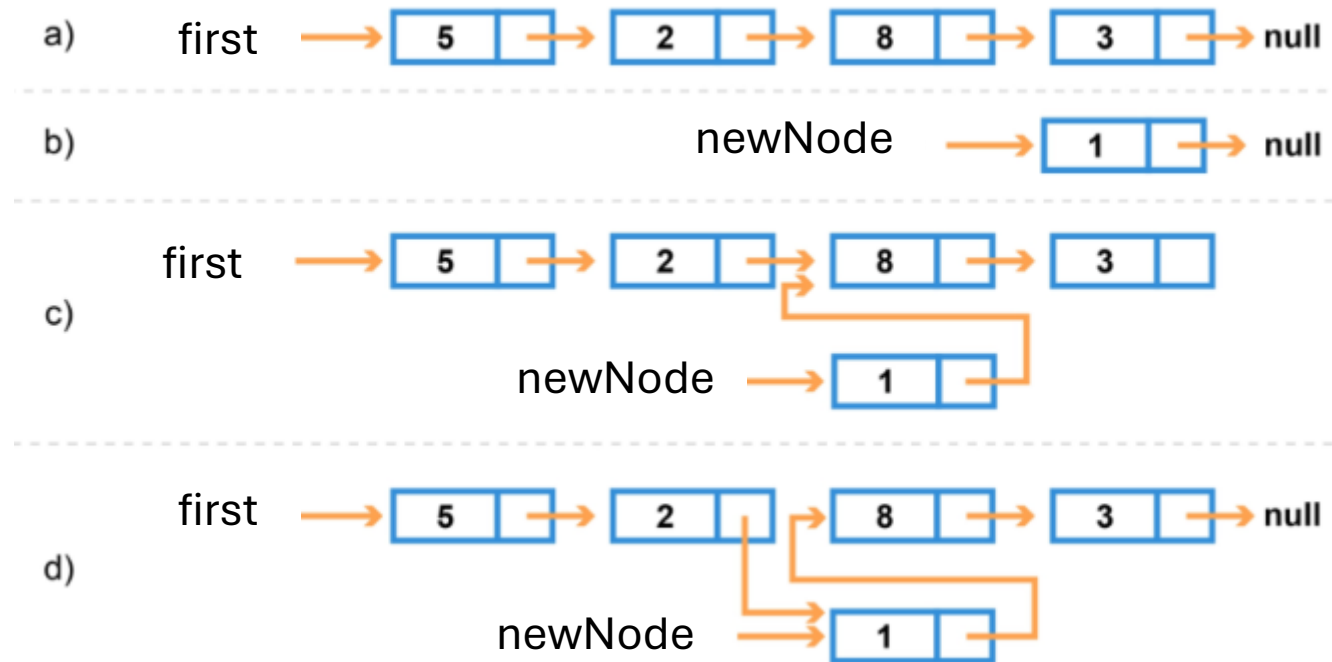
Simple Linked List

Add an element at the end



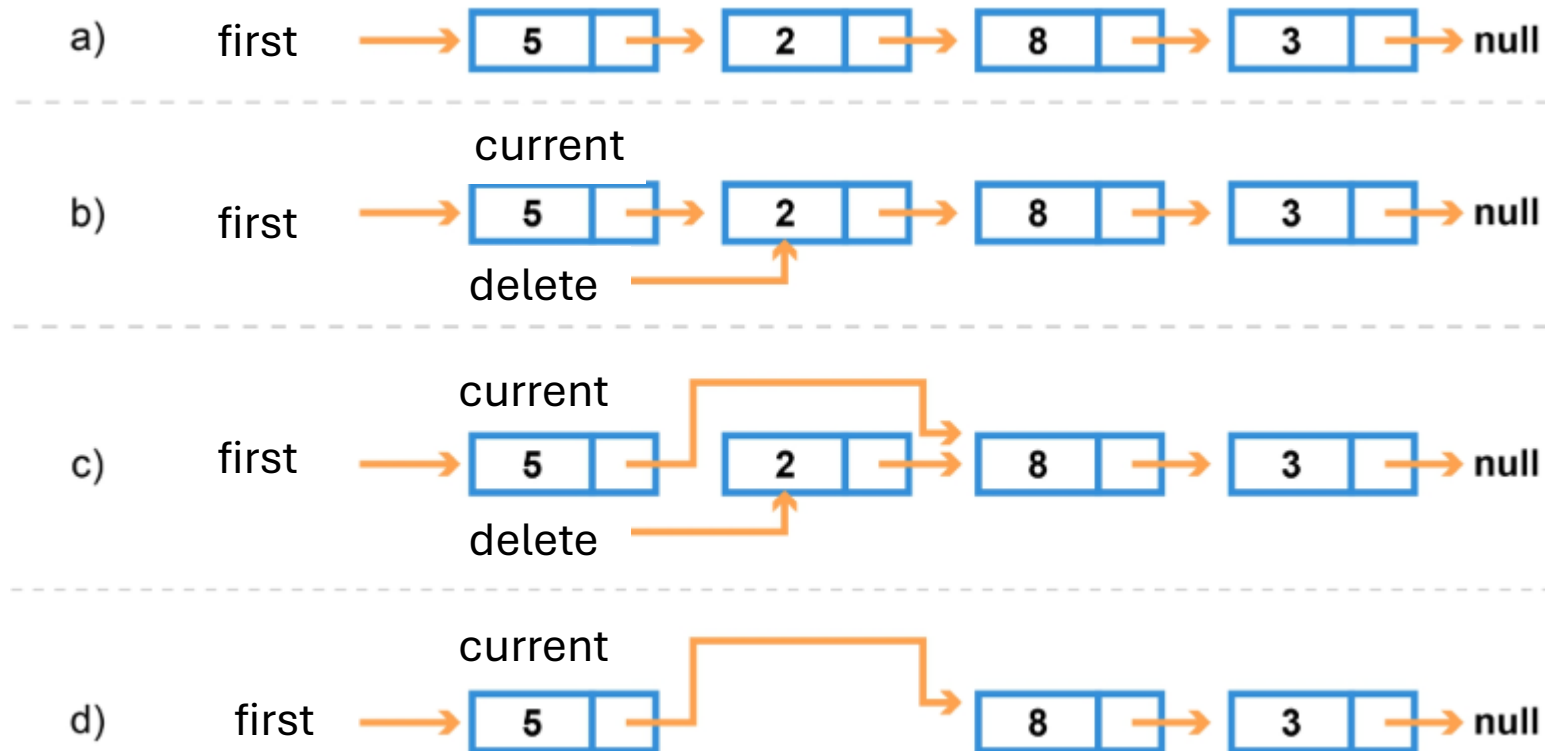
Simple Linked List

Add an element in the middle



Simple Linked List

Delete an element in the list



Simple Linked List

Insertion and deletion

*“If we know where a change is to be made, inserting or removing an item from a linked list does not require moving lots of items, and instead involves only a **constant number of changes to node links.**”*

Simple Linked List

Insertion and deletion

Add at the front

you keep a link to the front

$O(1)$

Add at the end

you keep a link to the last node

$O(1)$

Remove the first

head moves one step along

$O(1)$

Remove the last

you need the node BEFORE the last one

$O(n)$

*“In the classic linked list, where each node stores a link to its next node, **having a link to the last node provides no information about the next-to-last node.**”*

So removing the last item means walking the whole list to find its predecessor.

The List ADT

implemented as a simple double linked list

Double Linked List

Every node remembers the one before it



This is a **doubly linked list**

The price: every node now carries **two links instead of one**, and every insertion and removal has to keep **both of them consistent**.

Double Linked List

Every node remembers the one before it

Operation	Vector	Singly linked	Doubly linked
findKth(i) — the element at position i	$O(1)$	$O(i)$	$O(i)$
find(x) — search for a value No order assumed!	$O(N)$	$O(N)$	$O(N)$
Insert at the front	$O(N)$	$O(1)$	$O(1)$
Insert at the end	$O(1) *$	$O(1)$	$O(1)$
Remove the first	$O(N)$	$O(1)$	$O(1)$
Remove the last	$O(1)$	$O(N)$	$O(1)$
Remove a node you already point to	$O(N)$	$O(N)$	$O(1)$

* amortized — the vector doubles its capacity, so most push_backs are $O(1)$ and the occasional copy is paid off across them.

Every column walks the whole list in $O(N)$ to print it — but the vector walks contiguous memory, so its constant is smaller. Big-O does not see the cache.

References



M. A. Weiss, *Data Structures and Algorithm Analysis in C++*, 4th ed. Boston, MA, USA: Pearson, 2014.

V. M. de la Cueva Hernández, L. H. González Guerra, y E. G. Salinas Gurrión, *Estructuras de datos y algoritmos fundamentales*. Monterrey, México: Editorial Digital del Tecnológico de Monterrey, 2020.